

# Model-Driven Software Development

## Metamodel Programming

## Eternal Functions



Miguel Campusano

SDU Software Engineering

Previous class

Scope & Validation

# Summary

Scope

Validation

```
system University
```

```
entity Person
  attribute name is String
  attribute age is Number
  require age >= 0
```

```
entity Student
  - id is Number
  require age >= 15
```

```
entity PhDStudent
  require age >= 18
```

```
entity Course
  attribute title is String
  attribute ects is Number
  require ects > 0 && ects <= 3*10
```

```
Student is Person
PhDStudent -> Student
```

```
Person is PhDStudent
```

# Validation

We cannot add every constraints into the grammar

We do not want complex specifications

It may be impossible (Context-free grammars)

*A dog meows*

We *reduce* the possible metamodel instances

Bettini

*... do as little in the grammar and as much as possible in validation*

# Scope

Linking: uses of names/IDs to the element declarations

The scope of a declaration is the part of the program where it is *visible*

Scopes can be nested

- Scope and outer (parent) scope

- Look for the declaration in the inner scope,

- If not found, up one level, and so on

# Scope vs Validation

Forward reference

Use a symbol before its declaration

Visibility

public / private attributes

More examples?

*Loose scoping, Strict validation*

[PollEv.com/mica](https://poll-ev.com/mica)

# Model-Driven Software Development

## Metamodel Programming

## Eternal Functions



Miguel Campusano

SDU Software Engineering

# Summary

Metamodel programming

External functions

Project: Questions & Feedback



# Metamodel Programming

EBNF to Metamodel instance

# Metamodel Programming

Xtext infers the metamodel from the grammar specification

For each rule, a class is created\*

Alternatives ( `ClassA` | `ClassB` ) generates subtyping (subclasses)

Assignment generates instance variables

Use `returns` keyword to change class name

Actions ( { `ClassName` } ) to create hierarchy

Tree rewrite action (from lec06)

Cross-reference ( [ `ClassName` ] )

Cross-reference over two rules?

# Metamodel Programming

Xtext infers the metamodel from the grammar specification

A Xtext rule always generates a class

# Metamodel Programming

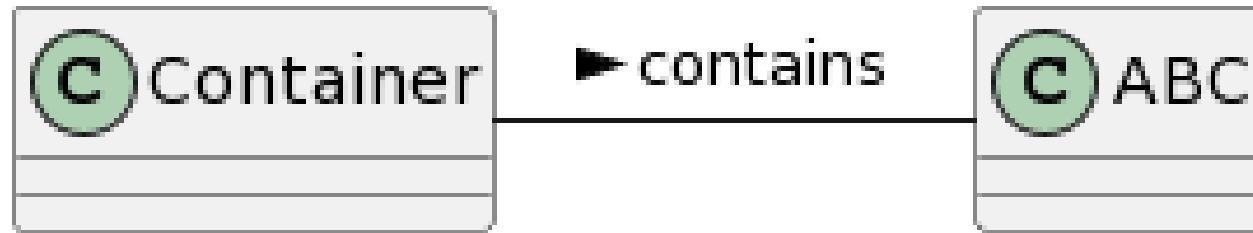
Xtext infers the metamodel from the grammar specification

Think: how can we create a class ABC using Xtext grammar?

# Metamodel Programming

Xtext infers the metamodel from the grammar specification

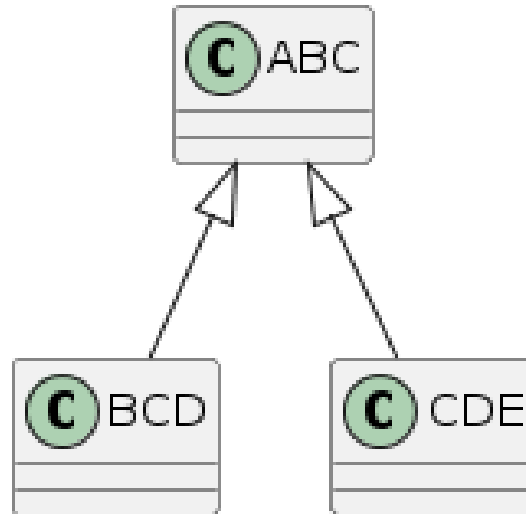
Think: how can we create associations



# Metamodel Programming

Xtext infers the metamodel from the grammar specification

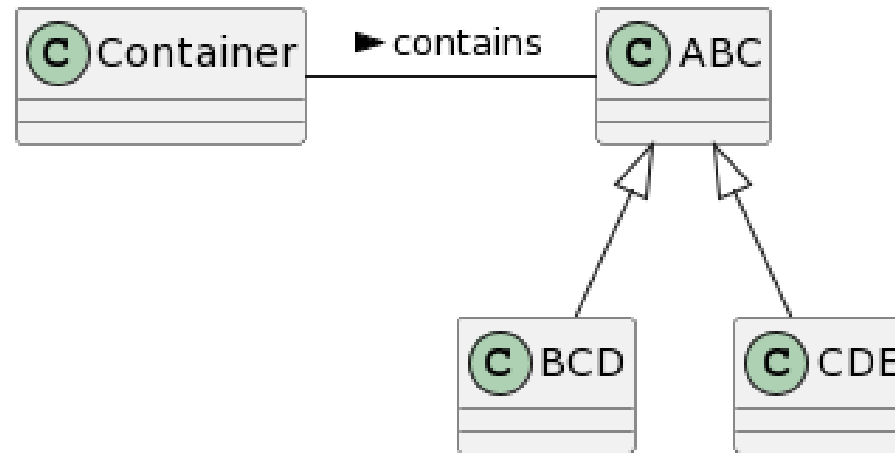
Think: how can we create subclasses?



# Metamodel Programming: in-class exercise

Xtext infers the metamodel from the grammar specification

Think: how can we do the following?



# Metamodel Programming

Xtext infers the metamodel from the grammar specification

For each rule, a class is created\*

Alternatives ( C l a s s A | C l a s s B ) generates subtyping (subclasses)

Assignment generates instance variables

Use returns keyword to change class name

Actions ( { C l a s s N a m e } ) to create hierarchy

**Tree rewrite action (from lec06): *Left recursion without left recursion***

Cross-reference ( [ C l a s s N a m e ] )

Cross-reference over two rules?



# External Functions

Complex function calls

# Class Description V3

**system** University

**entity** Person

**attribute** name **is** String

**attribute** age **is** Number

**require** age >= 0

**entity** Student

    - id **is** Number

**attribute** age >= 15

**entity** Teacher

**entity** Course

**attribute** title **is** String

**attribute** ects **is** Number

**require** ects > 0 && ects <= 3\*10

Student **is** Person

Teacher -> Person

Teacher **1--1** Course

Student **\*--\*** Course

S ::= 'system' ID E\* R\*

E ::= 'entity' ID (At|Q)\*

At ::= ('attribute' | '-') ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID<sub>E</sub> ('is' | '->') ID<sub>E</sub>

C ::= ID<sub>E</sub> ('1-1' | '\*-\*') ID<sub>E</sub>

Exp ::= ...

# Class Description V4: Complex functions

```
system University
external legalCPR(String) : Boolean
external complexExp(Number,Number) : Number
```

```
entity Person
  attribute name is String
  attribute age is Number
  attribute cpr is String
  require age >= 0
  require legalCPR(cpr)
```

```
...
```

```
require 0 <= complexExp(10,attr)
```

```
S ::= 'system' ID X* E* R*
```

```
X ::= 'external' ID '('(Type (',' Type)+)?')' : T
```

```
E ::= 'entity' ID (At|Q)*
```

```
At ::= ('attribute' | '-' ) ID 'is' T
```

```
Q ::= 'require' Exp
```

```
T ::= ...
```

```
R ::= I|C
```

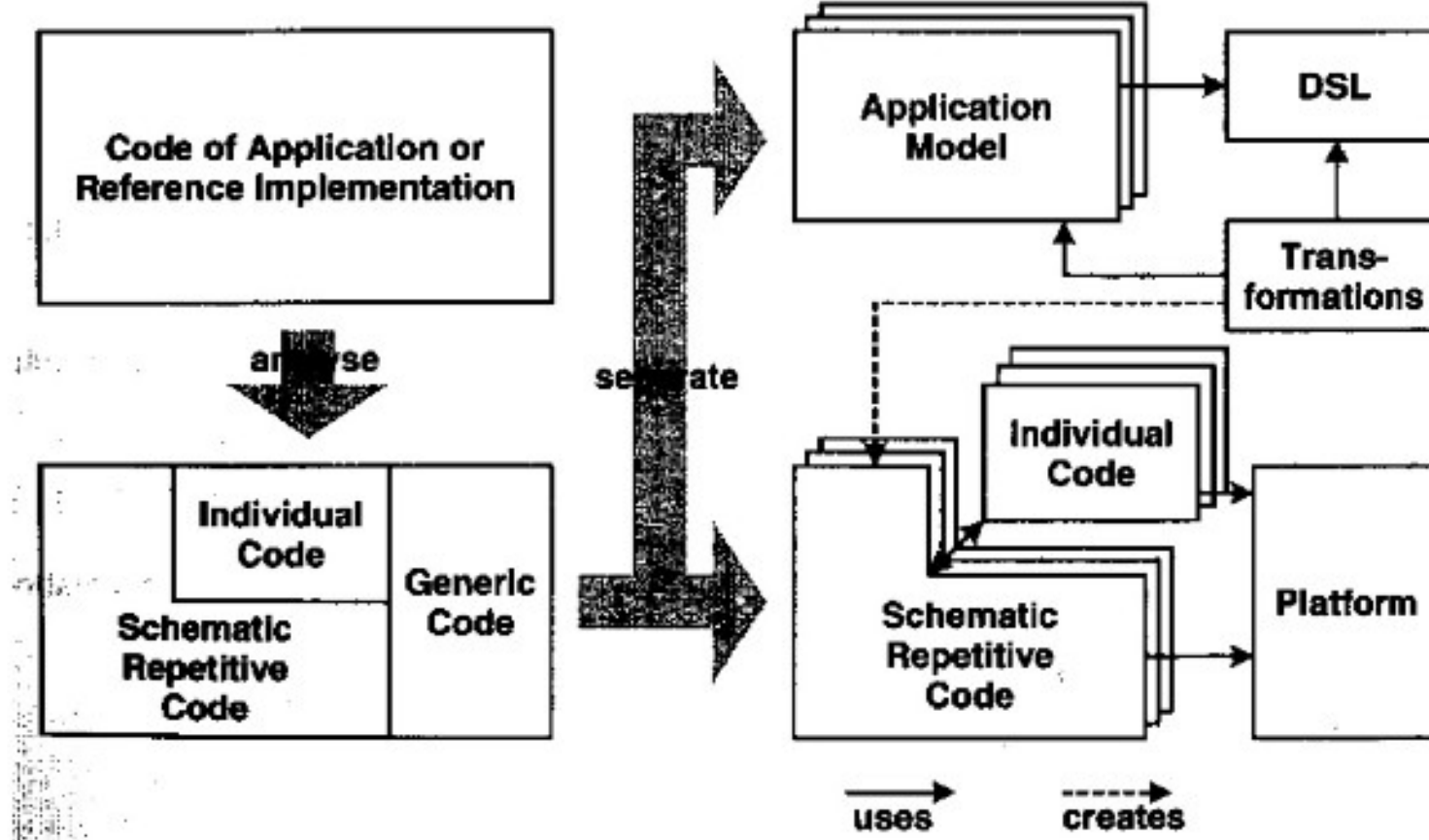
```
I ::= IDE ('is' | '->') IDE
```

```
C ::= IDE ('1-1' | '*-*') IDE
```

```
Exp ::= ... | Xc
```

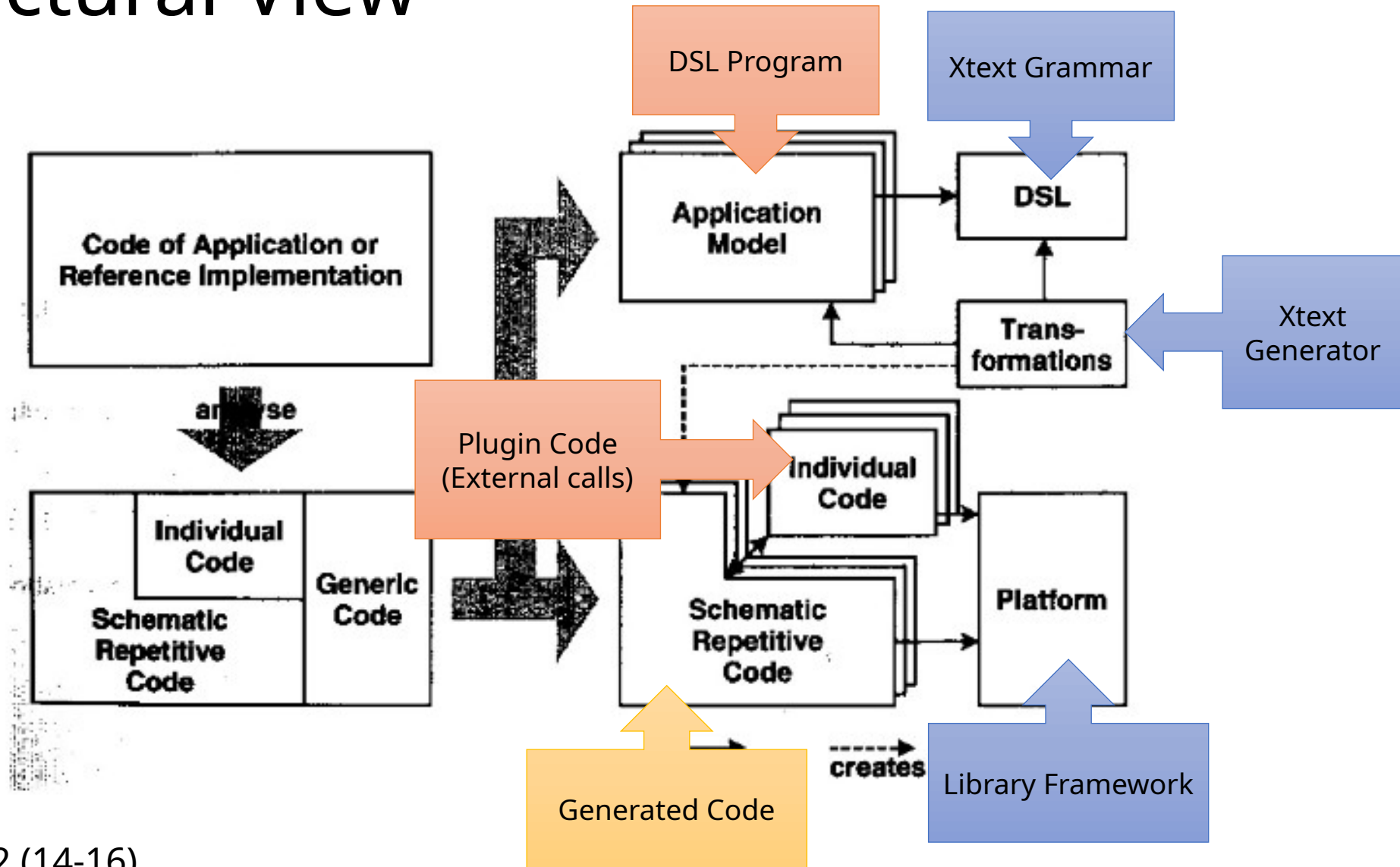
```
Xc ::= IDx '('(Exp (, Exp)+)?')'
```

# Architectural View



[MDSD]: Ch 2 (14-16)

# Architectural View



# Live Coding: extend grammar

```
system University
external legalCPR(String) : Boolean
external complexExp(Number,Number) : Number
```

```
entity Person
  attribute name is String
  attribute age is Number
  attribute cpr is String
  require age >= 0
  require legalCPR(cpr)
```

```
...
```

```
require 0 <= complexExp(10,attr)
```

```
S ::= 'system' ID X* E* R*
```

```
X ::= 'external' ID '('(Type (',' Type)+)?')' : T
```

```
E ::= 'entity' ID (At|Q)*
```

```
At ::= ('attribute' | '-' ) ID 'is' T
```

```
Q ::= 'require' Exp
```

```
T ::= ...
```

```
R ::= I|C
```

```
I ::= IDE ('is' | '->') IDE
```

```
C ::= IDE ('1-1' | '*-*') IDE
```

```
Exp ::= ... | Xc
```

```
Xc ::= IDx '('(Exp (, Exp)+)?')'
```

# External calls: how to

## Basic approach

- Generate code in a systematic way

- Clear indication of what is considered external code

- Describe a systematic way for the programmer to implement this code

## Possible realizations

- Dependency injection

- Interface

- Subclassing

- Static methods

- Edit generated code

# External calls: how to

Basic approach

- Generate code in a systematic way

- Clear indication of what is considered external code

- Describe a systematic way for the programmer to implement this code

Possible realizations

- Dependency injection** Object with interface is passed to main object

- Interface** Generate interface, programmer provides implementation

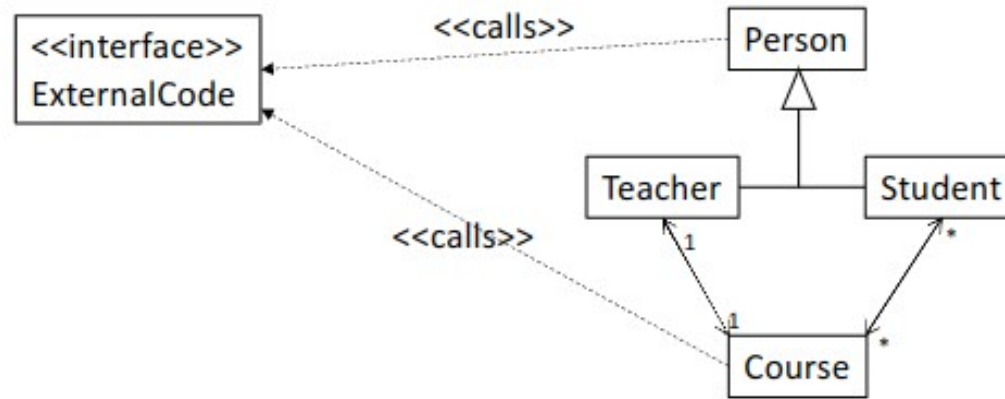
- Subclassing

- Static methods

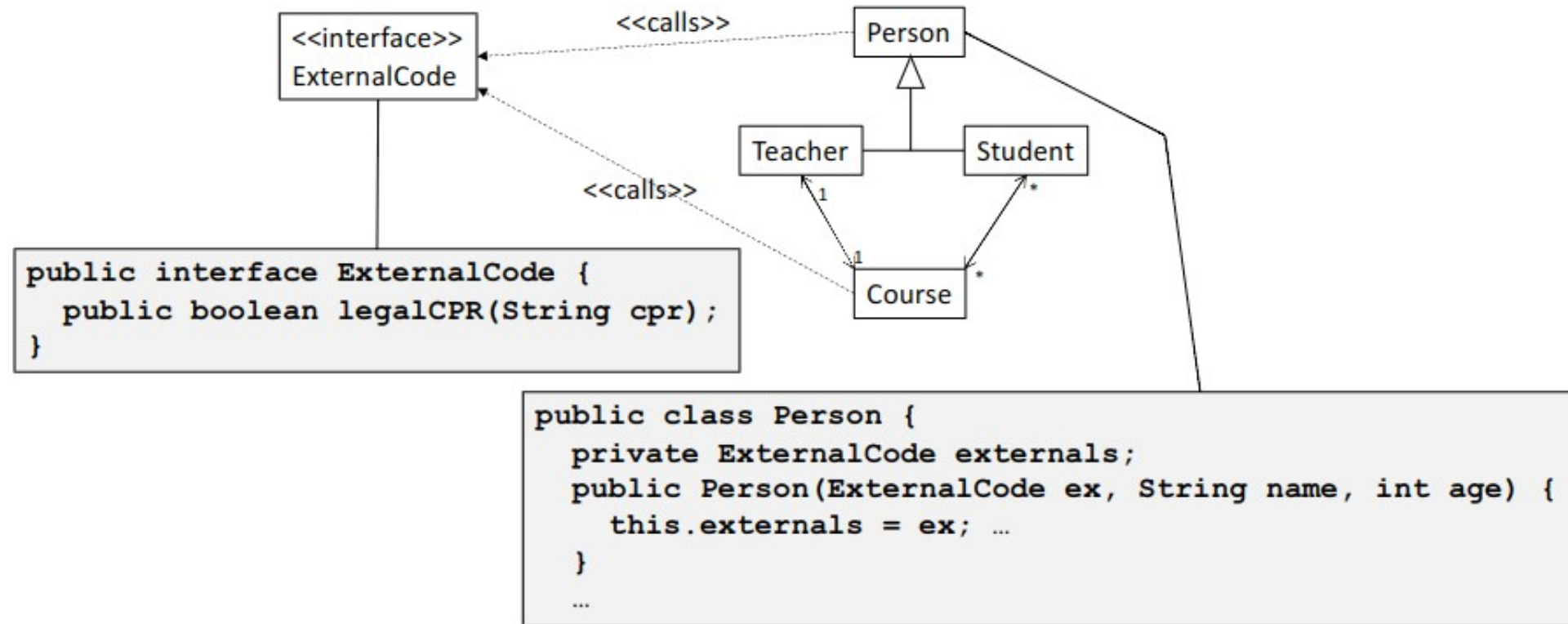
- Edit generated code



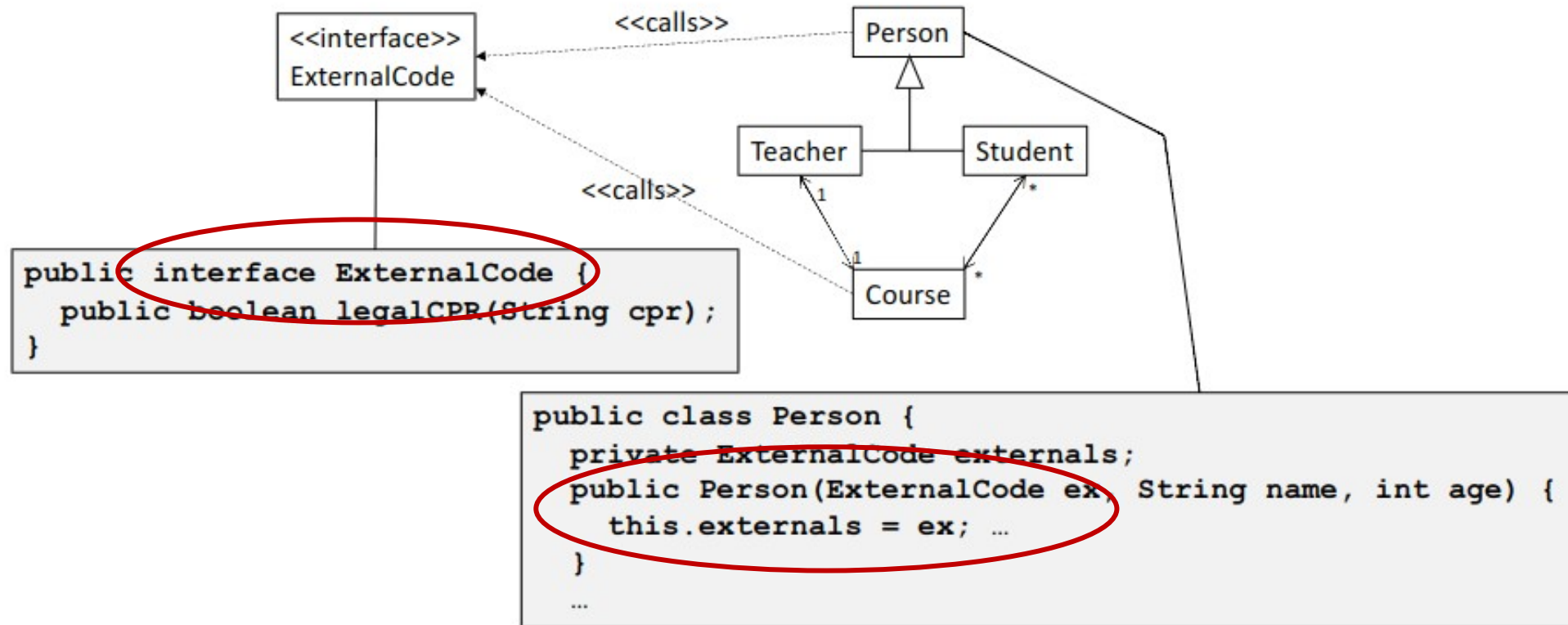
# External calls: Interface



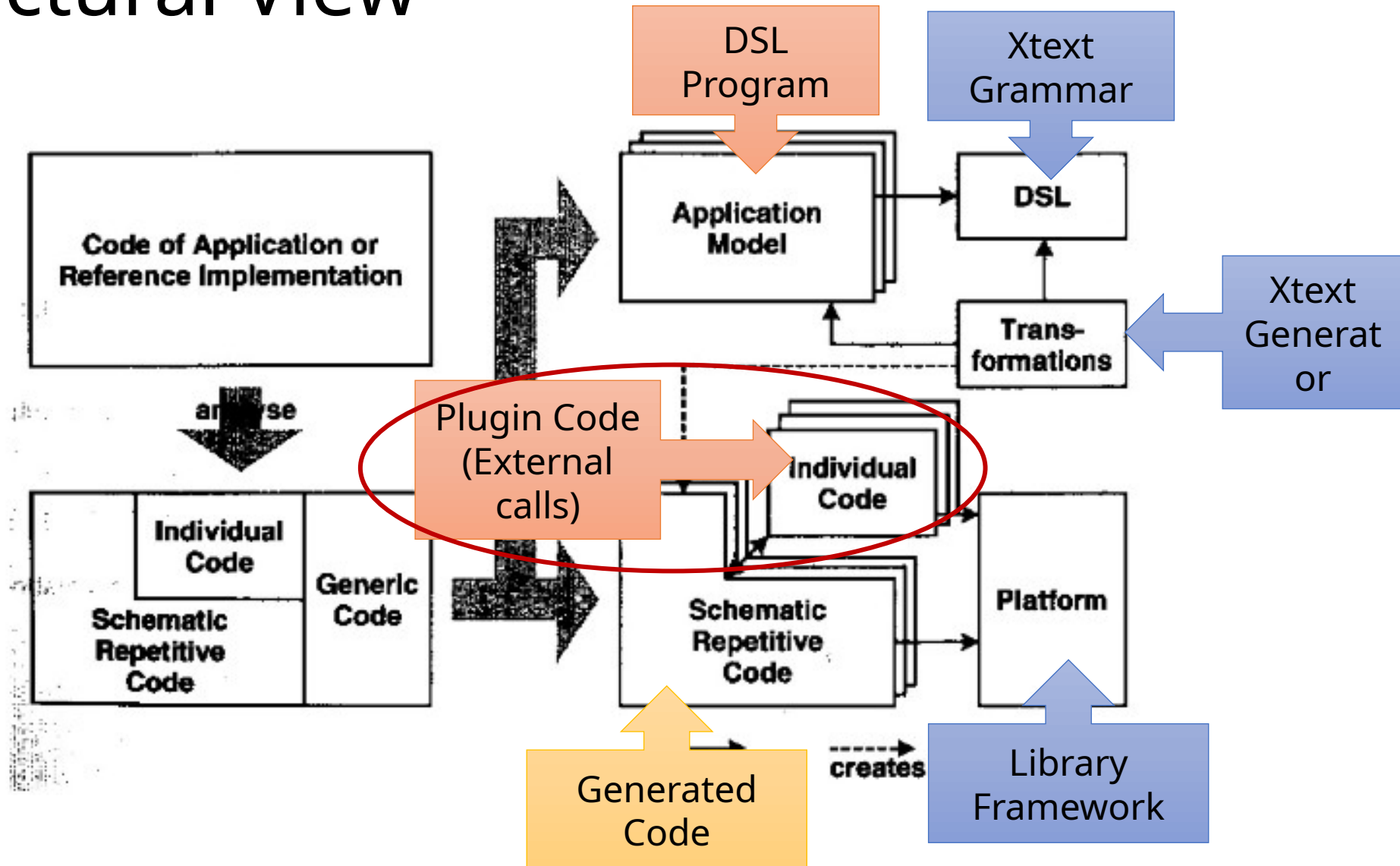
# External calls: Interface



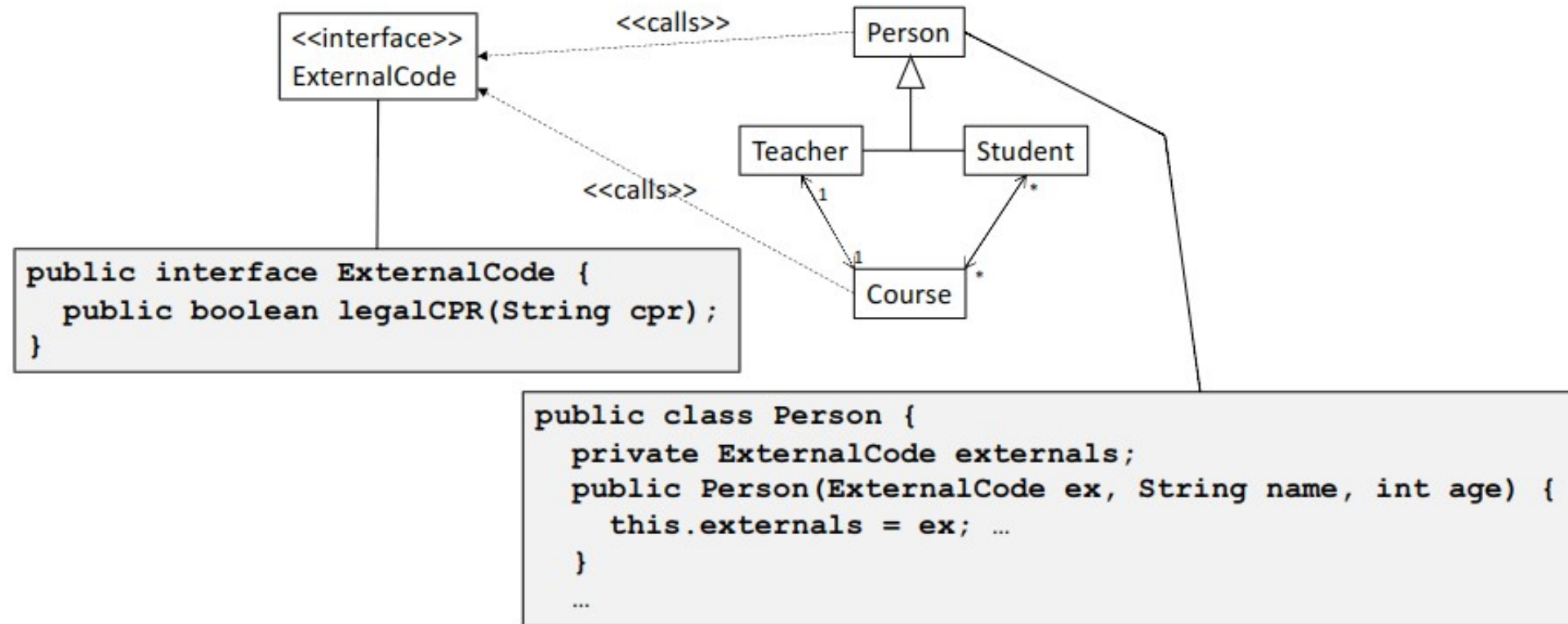
# External calls: Interface



# Architectural View



# Live coding: code generation attempt



# Projects

## Questions & Feedback

# Summary

# Summary

Metamodel programming

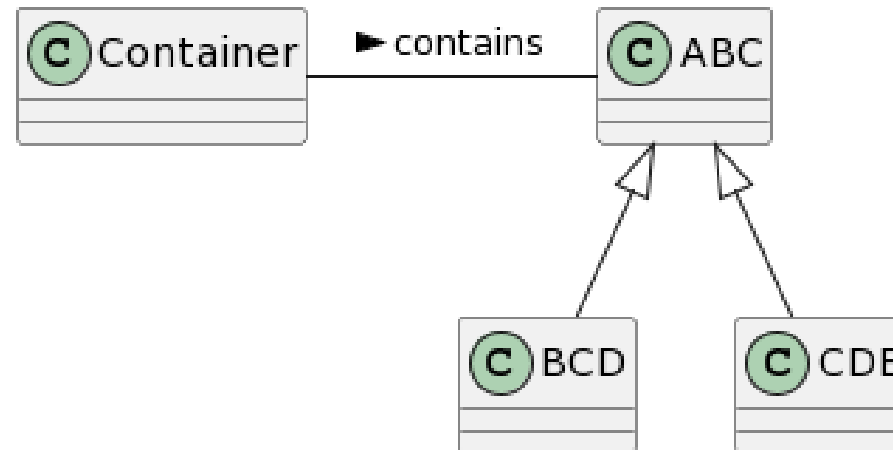
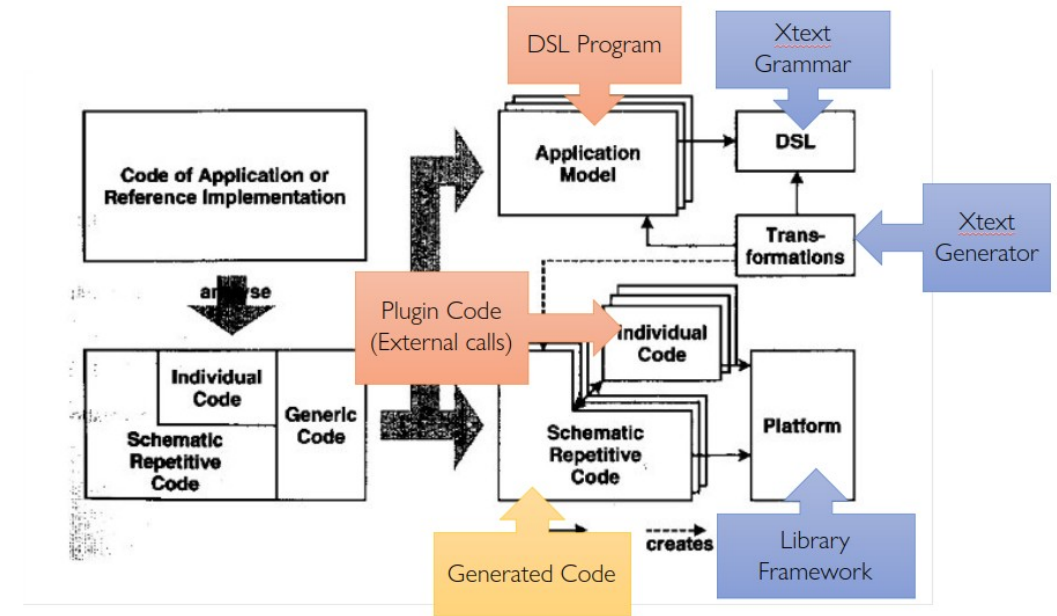
External functions

Project: Questions & Feedback

Next?

Programming eXperience

Type Systems





# Project Work

Week 8

Continue working on your group project

Research, choose *and work* on individual extension